

Topic 15: Major Practice Programs — Complete Code Walkthroughs

Program 1: Student Record System

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

const int MAX = 50;
string names[MAX];
int marks[MAX];
int count = 0;

char getGrade(int m) {
    if (m >= 80) return 'A';
    if (m >= 70) return 'B';
    if (m >= 60) return 'C';
    if (m >= 50) return 'D';
    return 'F';
}

void addStudent() {
    if (count >= MAX) { cout << "Record full.\n"; return; }
    cout << "Name: "; cin >> names[count];
    cout << "Marks: "; cin >> marks[count];
    count++;
    cout << "Student added.\n";
}

void displayAll() {
    cout << "\n--- Student Records ---\n";
    cout << "Name\t\tMarks\tGrade\n";
    for (int i = 0; i < count; i++)
        cout << names[i] << "\t\t" << marks[i] << "\t" << getGrade(marks[i]) <<
        "\n";
}

void saveToFile() {
```

```

        ofstream file("students.txt");
        for (int i = 0; i < count; i++)
            file << names[i] << " " << marks[i] << " " << getGrade(marks[i]) <<
"\n";
        file.close();
        cout << "Saved to students.txt\n";
    }

int main() {
    int choice;
    do {
        cout << "\n1.Add 2.Display 3.Save 0.Exit\nChoice: ";
        cin >> choice;
        if (choice == 1) addStudent();
        else if (choice == 2) displayAll();
        else if (choice == 3) saveToFile();
    } while (choice != 0);
    return 0;
}

```

Program 2: Menu-Driven Calculator with History

```

#include <iostream>
#include <fstream>
using namespace std;

const int MAX_HISTORY = 100;
string history[MAX_HISTORY];
int histCount = 0;

void record(string entry) {
    if (histCount < MAX_HISTORY)
        history[histCount++] = entry;
}

void showHistory() {
    cout << "\n--- Calculation History ---\n";
    for (int i = 0; i < histCount; i++)
        cout << i + 1 << ". " << history[i] << "\n";
}

void saveHistory() {
    ofstream file("history.txt", ios::app);
    for (int i = 0; i < histCount; i++)

```

```

        file << history[i] << "\n";
    file.close();
    cout << "History saved to history.txt\n";
}

int main() {
    double a, b;
    int choice;
    do {
        cout << "\n1.Add 2.Subtract 3.Multiply 4.Divide 5.History 6.Save
0.Exit\nChoice: ";
        cin >> choice;
        if (choice >= 1 && choice <= 4) {
            cout << "Enter two numbers: ";
            cin >> a >> b;
        }
        double result = 0;
        string entry;
        if (choice == 1) { result = a + b; entry = to_string(a) + " + " + to_string(b) + " = " + to_string(result); }
        else if (choice == 2) { result = a - b; entry = to_string(a) + " - " + to_string(b) + " = " + to_string(result); }
        else if (choice == 3) { result = a * b; entry = to_string(a) + " * " + to_string(b) + " = " + to_string(result); }
        else if (choice == 4) {
            if (b == 0) { cout << "Cannot divide by zero.\n"; continue; }
            result = a / b;
            entry = to_string(a) + " / " + to_string(b) + " = " + to_string(result);
        }
        if (choice >= 1 && choice <= 4) {
            cout << "Result: " << result << "\n";
            record(entry);
        }
        else if (choice == 5) showHistory();
        else if (choice == 6) saveHistory();
    } while (choice != 0);
    return 0;
}

```

Program 3: Bank Account Management System (OOP)

```

#include <iostream>
#include <string>

```

```
using namespace std;

class BankAccount {
private:
    string owner;
    double balance;

public:
    BankAccount(string name, double startBalance) {
        owner = name;
        balance = startBalance;
        cout << "Account created for " << owner << " with balance: " << balance
<< "\n";
    }

    void deposit(double amount) {
        if (amount <= 0) { cout << "Invalid amount.\n"; return; }
        balance += amount;
        cout << "Deposited " << amount << ". New balance: " << balance << "\n";
    }

    void withdraw(double amount) {
        if (amount <= 0) { cout << "Invalid amount.\n"; return; }
        if (amount > balance) { cout << "Insufficient funds.\n"; return; }
        balance -= amount;
        cout << "Withdrew " << amount << ". New balance: " << balance << "\n";
    }

    void displayBalance() const {
        cout << owner << "'s balance: " << balance << "\n";
    }

    string getOwner() const { return owner; }
};

int main() {
    BankAccount acc1("Ahmad", 5000);
    BankAccount acc2("Sara", 3000);

    acc1.deposit(1500);
    acc1.withdraw(800);
    acc1.displayBalance();

    acc2.deposit(2000);
    acc2.withdraw(5000); // should fail
}
```

```
acc2.displayBalance();

cout << "\n--- Final Balances ---\n";
acc1.displayBalance();
acc2.displayBalance();

return 0;
}
```